Making the icon sets

build_iconsets.py makes the icon sets in data/iconsets/<Set>/ and their M.U.G.E.N extras in data/engines/mugen/iconsets/<Set>/. The files it makes are in the repository, so the program works without running it. Run it again after you change one of its tables.

Set	Look	Pictures
Nuvola	shiny color icons (the default preset)	Nuvola 1.0 PNGs, pictures put together from them, and our own SVG drawings
Flat-Dark	flat outline icons, light shapes for dark styles	Tabler Icons and Lucide SVGs, recolored, with badges, and our own SVG drawings
Flat-Light	the same with dark shapes for light styles	same as Flat-Dark
Classic, Ultimate	the Nuvola set with their own pictures for the icons those presets change (about a third, like Fighter Factory)	Tango PNGs and pictures put together from them for those, and everything else points to ../Nuvola/...

How it works

- LAYOUT_GLOBAL and LAYOUT_MUGEN give every icon name in the program a file, like common/save (one file can be used by several names). In the M.U.G.E.N layout a * at the start keeps the file in the global set, and the extra set's iconset.xml then uses the program's * redirect.
- NUVOLA, FLAT, CLASSIC and ULTIMATE say how each file is made (Tg("actions/list-add") is a Tango picture drawn from its SVG, and works like Nv):
    - Nv("actions/filesave", badges=(B(Nv("apps/clock")),)): a Nuvola picture (taken at 48 px, the biggest size of most action icons, or made smaller from 128 px). It can have its hue changed, be stacked, or get smaller pictures over a corner. Saved as PNG.
    - Draw(fn): one of our drawings in drawings.py, saved as SVG. Letters and numbers use the outline shapes from Tabler Icons.
    - Flat("device-floppy", badge="plus"): a Tabler outline icon ("lucide:name" for Lucide) in the set's shape color, with a colored badge if you want, saved as SVG.
- Every set folder gets an iconset.xml, a Sources.pdf that lists each file with the names that use it and the pictures it's made from, and copies of the licenses of those pictures.
- required_names.json lists every icon name the program uses (236 global, 83 M.U.G.E.N), and the builder won't write a set that's missing one.

The archives with the original pictures are downloaded once into tools/iconsets/.cache/ (git ignores it) and checked against the SHA-256 values in sources.py:

Source	Version	License	Archive
Nuvola icon theme (David Vignoni)	1.0	LGPL-2.1	nuvola-1.0.tar.gz (the original icon-king.com file, from the Internet Archive)

Source	Version	License	Archive
Tabler Icons	3.48.0	MIT	npm @tabler/icons
Lucide	1.51.0	ISC	npm lucide-static
Tango Icon Library	0.8.90	Public domain	Debian's copy of the release archive

No picture is taken from another program's install.

Running it

```
python -m venv .venv
.venv/Scripts/python -m pip install -r tools/iconsets/requirements.txt   # bin/python on Linux/macOS
.venv/Scripts/python tools/iconsets/build_iconsets.py           # make every set again
.venv/Scripts/python tools/iconsets/build_iconsets.py --check    # check the files in the repository
.venv/Scripts/python tools/iconsets/build_iconsets.py --sets Nuvola,Flat-Dark
```

It draws with **Pillow** (resizing, putting pictures together, saving PNG) and **Qt's SVG renderer through PySide6** (to draw our drawings when they go over Nuvola pictures). To get the exact same bytes you need the versions in requirements.txt. With other versions the pictures look the same, but the PNG bytes can be different.

--deny-hashes FILE makes the build stop when the SHA-256 of a file it makes is in FILE (one per line), for example a list of files that must not be shared.

Checking

check_iconsets.py only needs Python itself. It checks that the full sets (Nuvola, Flat-Dark, Flat-Light, or every set with --all) have every name the program needs, and that every file they point to is there and is a good PNG or SVG. Each folder also needs Sources.pdf and a license file. It runs as the iconsets_check test, and tests/core/ThemeDataTest.cpp loads the same sets through OFS::IconSet and draws every icon.

The pictures for the interface styles are drawn by tools/styles/build_style_images.py.